BIG DATA (T) - FINAL PROJECT - 2026

Cross-Mission Exoplanet Classifier for Combined Kepler and TESS Data

Separating real planets from false positives with four gradient-tree and ensemble models

Mohamad Valdi Ananda Tauhid - 5025221238

Dept. of Informatics, Institut Teknologi Sepuluh Nopember

ABSTRACT Transit surveys flag far more candidate signals than human vetters can confirm, and most of those signals turns out not to be planets at all. This project trains a classifier to do that first-pass triage automatically. I merge the labeled signals from the Kepler and TESS missions into one 9,725-row dataset, built on seven physical features the two missions share plus two ratios I engineered, then train four models on it: Random Forest, XGBoost, LightGBM, and CatBoost. All four land around 0.955 ROC-AUC on a held-out test set, with XGBoost a hair ahead on F1 (0.872) and CatBoost on average precision (0.928). The surprising result is that every model reads Kepler rows better than TESS rows, which the per-mission analysis traces to the Kepler-heavy training mix and the noisier stars TESS observes. Planet radius and the transit duty cycle are the two most informative features, consistently, across every model.

1 Introduction

Transit photometry spots a planet by watching for the small dip in a star's brightness when the planet passes in front of it[1]. Kepler used this to stare at about 150,000 stars continuously for four years[2]; its successor TESS now sweeps almost the whole sky in 27-day sectors[3]. Between them the two missions have turned up tens of thousands of transit-like signals.

The problem is that a planet is not the only thing that dims a star on a regular schedule. Eclipsing binary stars, light from a background source bleeding in, and plain instrument systematics all produce dips that look almost identical to a real transit[1]. So the false-positive rate is high: in the Kepler cumulative catalog I use here, about 58% of the labeled signals turned out not to be planets. And every candidate that does survive the automated checks still needs follow-up spectroscopy or imaging before anyone can call it confirmed, which runs to months per target[4]. A model that ranks thousands of archived signals by how planet-like they look lets a vetting team point its limited telescope time at the ones most likely to pay off.

Most published vetting models train on a single mission[5][6]. I combined both instead. Kepler and TESS watch different stellar populations over different orbital-period ranges, so a model that has to fit both is pushed to learn the signal characteristics that actually generalize rather than one instrument's quirks. The caveat is that the two catalogs name their columns and label their dispositions differently, which I handle in preprocessing.

Objectives

- Merge the Kepler KOI and TESS TOI catalogs into one single consistently labeled and feature-aligned dataset.
- Engineer physically motivated features that help separate planets from diluted eclipsing binaries.
- Train and benchmark four ensemble classifiers across six metrics, cross-validation stability, and training cost.
- Diagnose where and why the models fail, with attention to per-mission differences.

2 Dataset and Exploratory Analysis (EDA)

I pulled both catalogs from the NASA Exoplanet Archive TAP service[7]: the Kepler cumulative Object of Interest table and the TESS Object of Interest (TOI) list, both as CSV. I only keep the rows that have a settled disposition. The uncategorized candidates (`CANDIDATE` in Kepler, `PC` and `APC` in TESS) have no ground-truth label, so they had to go.

SOURCE CATALOG	TOTAL ROWS	LABELED ROWS USED	POSITIVE LABEL MAPS FROM
Kepler KOI Cumulative	9,564	7,327	`CONFIRMED`
TESS TOI List	7,931	2,398	`CP` , `KP`
Combined (cleaned)	—	**9,725**	negatives: `FALSE POSITIVE` , `FP` , `EB`

Table 1. After remapping columns, encoding labels as planet (1) / false positive (0), and dropping rows with missing or non-finite feature values, 9,725 rows remain.

Features

After doing that, I crossmatched the TESS column names onto Kepler's convention so the two catalogs can share one single schema. Seven base features survive in both missions. I left out the transit impact parameter (`koi_impact`) because TESS does not report it, and filling half the rows with a placeholder would have injected a constant the model could easily mistake for real signal.

FEATURE	PHYSICAL QUANTITY	UNIT
`koi_period`	Orbital period	days
`koi_prad`	Planet radius	Earth radii
`koi_depth`	Transit depth (fraction of flux blocked)	ppm
`koi_duration`	Transit duration	hours
`koi_steff`	Stellar effective temperature	K
`koi_srad`	Stellar radius	Solar radii
`koi_insol`	Insolation flux on the planet	Earth flux
`radius_ratio` *(engineered)*	Planet radius / stellar radius, in matched units	ratio
`transit_duty_cycle` *(engineered)*	Transit duration / orbital period	ratio

Table 2. Nine features total. The two engineered ratios flag geometries, normal for diluted eclipsing binaries, is where a stellar companion mimics a transit but the implied size and timing are inconsistent with a planet.

What the data looks like

After we cleaned the set, it leans toward false positives, 5,684 of them against 4,041 confirmed planets, a 58/42 split that lines up with the survey reality that most transit-like signals are usually not planets. Correlations between features are mostly weak, which is what you want for tree ensembles since they prefer non-redundant inputs. The two strong off-diagonal values are there by construction: `radius_ratio` tracks `koi_prad` (r = 0.68) because it is computed from it, and insolation correlates with stellar radius (r = 0.56) through the physics of received flux.

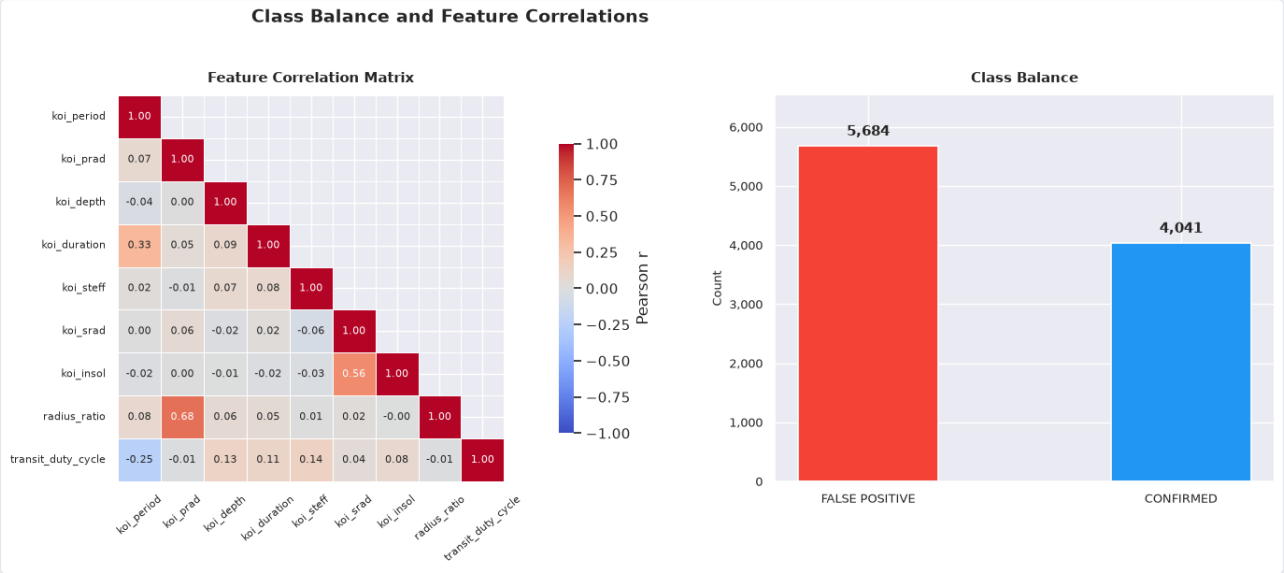

Figure 1. Left: pairwise Pearson correlations among the nine features (lower triangle). Right: class balance, 5,684 false positives vs 4,041 confirmed planets.

Splitting each feature by class shows where the separating power actually lives. Planet radius, transit depth, and the two engineered ratios pull the classes apart cleanly: confirmed planets bunch up at smaller radii and tighter duty cycles, while false positives spread out into the large-radius, long-duration tail you would expect from an eclipsing binary. Stellar temperature and stellar radius overlap almost completely, so on their own they tell the model very little.

Figure 2. Density of each feature split by class (blue = confirmed, red = false positive). Five features use a log x-axis because they span several orders of magnitude. Wide separation between the two colors predicts a feature's usefulness to the model.

3 Methodology

The whole thing is one linear pipeline, from two raw catalogs to a benchmarked set of four models.

Load	Align	Engineer	Split	Train + Compare
Kepler KOI + TESS TOI from NASA TAP	Remap columns, encode 0/1 labels, drop unvetted rows	radius_ratio, transit_duty_cycle	80/20 stratified, scale	4 models, 6 metrics, 5-fold CV

Every model sits inside a scikit-learn `Pipeline` whose first step is a `StandardScaler` [8]. I fit the scaling on the training fold only, so nothing from the test set leaks back into the model. Tree ensembles usually do not really need

scaled inputs, but keeping the scaler in makes the pipeline uniform across all four models and costs nothing. I split the data 80/20, stratified on the label, which gives 7,780 training rows and 1,945 test rows that hold the same 58/42 class ratio.

I picked four ensemble methods because they dominate tabular classification, and because each one can handle the class imbalance through a built-in weighting argument instead of resampling:

MODEL	FAMILY	IMBALANCE HANDLING
Random Forest[9]	Bagged decision trees	`class_weight='balanced'`
XGBoost[10]	Gradient boosting	`scale_pos_weight` (neg/pos ratio)
LightGBM[11]	Histogram gradient boosting	`is_unbalance=True`
CatBoost[12]	Ordered gradient boosting	`auto_class_weights='Balanced'`

Table 3. All four use 200 trees (boosters) and shared depth and learning-rate settings, so the comparison reflects the algorithms rather than wildly different capacities.

I judge the models on six metrics. Accuracy on its own is misleading under imbalance, so I lean on F1 and ROC-AUC, and add average precision and the precision-recall curve because the positive class is the minority here. Recall is the one that really matters in this domain: a missed planet (a false negative) costs you a discovery, while a false alarm only costs a bit of follow-up time. I check stability with stratified 5-fold cross-validation, and record wall-clock training time as the practical cost.

4 Results and Analysis

The four models finish within a hair of each other. In the held-out test set, every one of them lands near 0.886 accuracy and 0.955 ROC-AUC. XGBoost takes the top F1 (0.8717), but only by half a thousandth over Random Forest, and CatBoost edges out the best average precision. Differences this small most of the time do not mean much in practice, but its definelty a knowledge worth knowing. On this data, the signal lives in the features, not in which ensemble you reach for.

MODEL	ACCURACY	PRECISION	RECALL	F1	ROC-AUC	AVG PREC
XGBoost	**0.8859**	**0.8178**	**0.9332**	**0.8717**	**0.9571**	**0.9272**
Random Forest	0.8869	0.8267	0.9208	0.8712	0.9548	0.9244
CatBoost	0.8853	0.8197	0.9282	0.8706	0.9568	0.9275
LightGBM	0.8812	0.8167	0.9208	0.8656	0.9552	0.9209

Table 4. Test-set metrics, sorted by F1. The highlighted row is the best F1. Recall sits well above precision for every model, a direct effect of the balanced class weighting, which favors catching planets over avoiding false alarms.

Figure 3. Six metrics side by side. The bars are nearly flat across models within each metric. Recall (~0.92-0.93) exceeds precision (~0.82) everywhere, the fingerprint of class-balanced training.

The ROC and precision-recall curves tell the same story by eye: four lines sitting almost on top of one another, all well above the no-skill baselines. At a 0.42 positive prevalence the precision-recall curves stay above 0.9 precision across most of the recall range, and that is the more honest view of how the models do on the minority class.

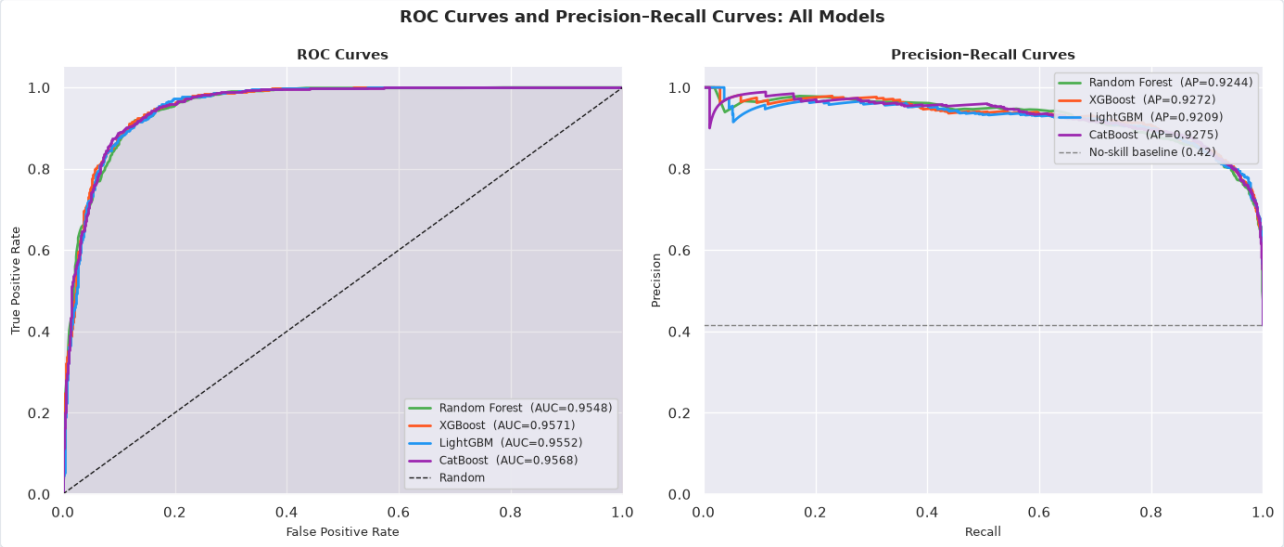

Figure 4. ROC curves (left) and precision-recall curves (right). All four models cluster tightly; the dashed lines mark a random classifier and the 0.42 no-skill precision baseline.

Cross-validation confirms the ranking is not an artifact of one lucky split. Across five stratified folds the F1 scores stay inside a 0.85-0.88 band for every model, and the boxes overlap. Random Forest has the widest spread, LightGBM and CatBoost the tightest, but none of them is what I would call unstable.

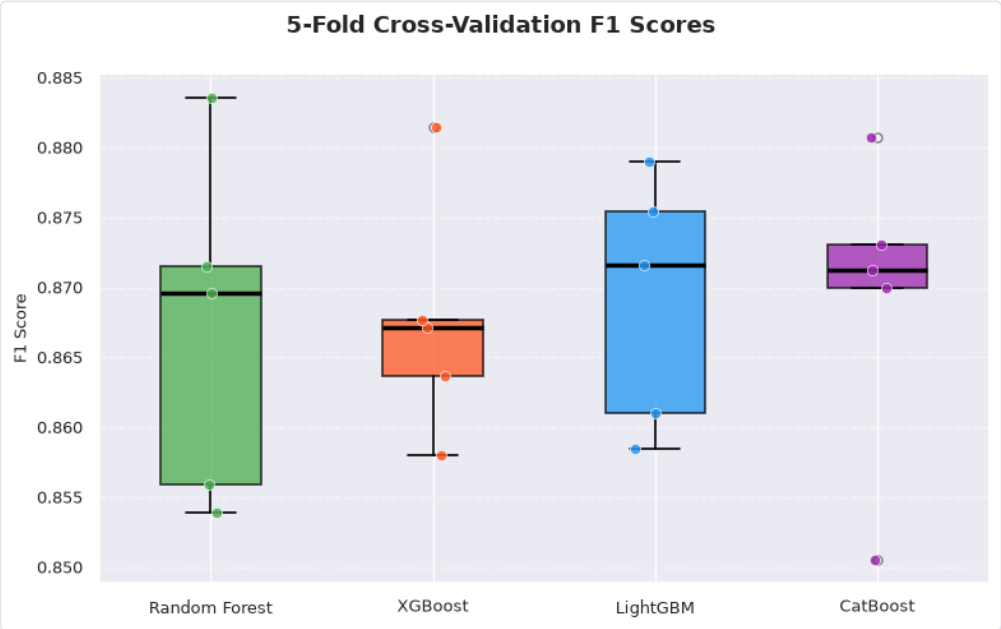

Figure 5. 5-fold cross-validation F1 distributions. Overlapping boxes mean the models are statistically indistinguishable on this data; the spread, not the median, is what separates them.

Where the models genuinely part ways is training cost. LightGBM finishes in 0.34 s and CatBoost in 0.76 s, while XGBoost takes a full 30 s, almost two orders of magnitude slower for no accuracy gain I can measure. If I were retraining this often, or scaling it to the whole unvetted archive, LightGBM is the obvious default: the best speed, with accuracy sitting inside the noise of the leader.

Figure 6. Wall-clock training time. XGBoost is the clear outlier; the histogram-based methods are dramatically cheaper at equal accuracy.

What the models pay attention to

Feature importance comes out consistent across all four algorithms. Planet radius and the engineered transit duty cycle are the two strongest signals, with the radius ratio just behind. Both of the ratios I added by hand land in the top three, which is about as direct as evidence gets that the feature engineering did real work and was not just echoing the raw columns. Stellar temperature and stellar radius come last, which matches the heavy class overlap they showed back in Figure 2.

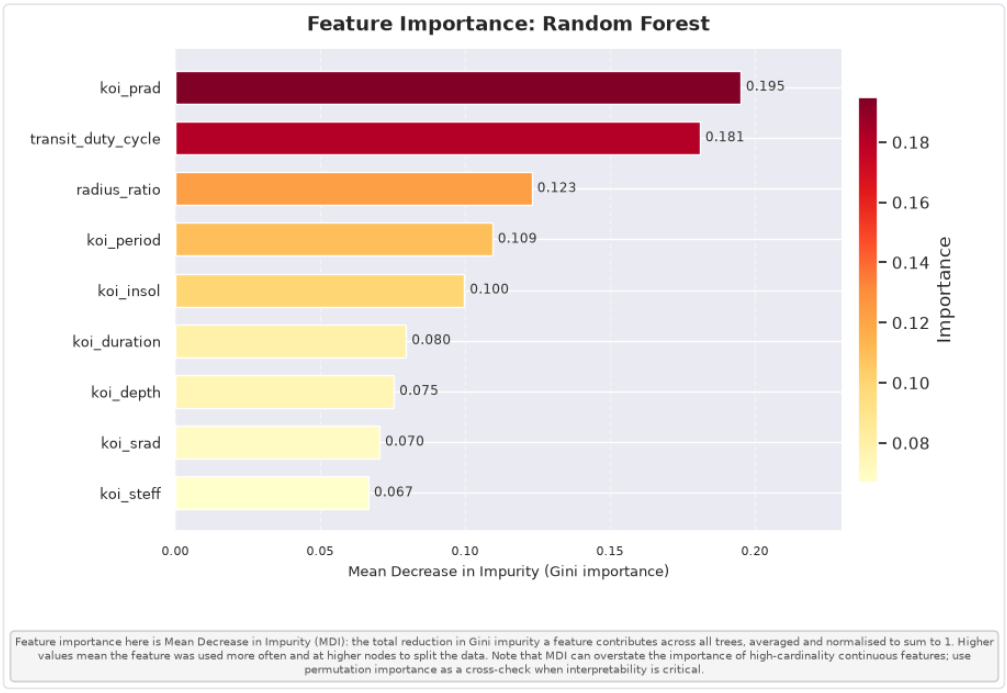

Figure 7. Random Forest feature importance (mean decrease in impurity). The engineered `transit_duty_cycle` and `radius_ratio` rank second and third. MDI can flatter high-cardinality continuous features, so it is read here as a ranking rather than an exact weight.

Per-mission breakdown

Splitting the test set by telescope is where the one real weakness shows up. Every model scores noticeably higher on Kepler rows than on TESS rows. Taking Random Forest as the reference, Kepler reaches 0.967 ROC-AUC against 0.908 for TESS, and that gap holds in F1 across all four models. Two things are driving it. About three-quarters of the training data is Kepler, so the model is mostly fit to Kepler-shaped signals in the first place. And TESS tends to target brighter, more active stars on shorter orbital periods, so its false positives sit in a harder corner of the feature space.

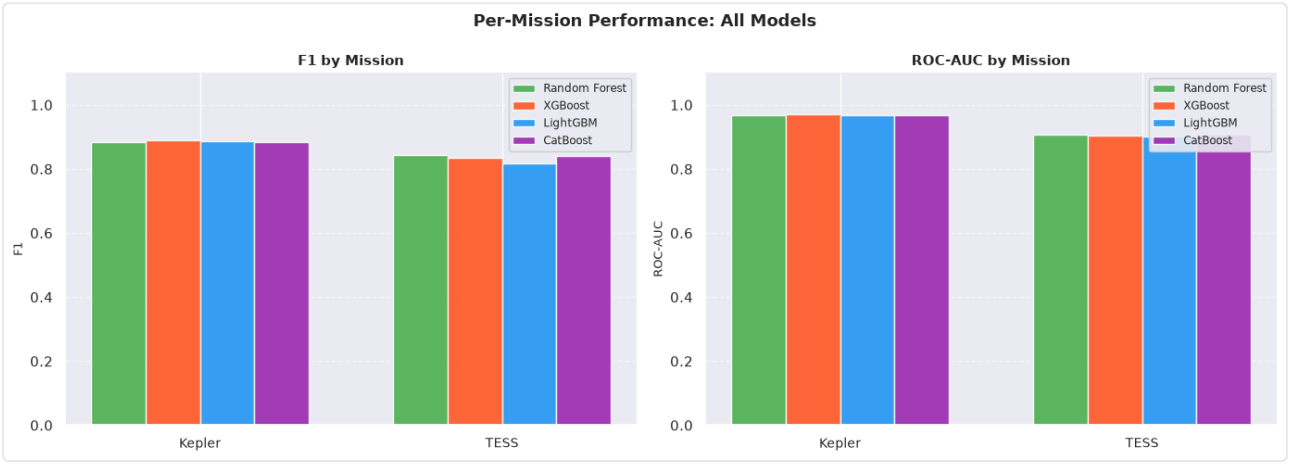

Figure 8. F1 (left) and ROC-AUC (right) split by mission. The Kepler-to-TESS drop is consistent across all four models, pointing to a data-distribution cause rather than a model-specific one.

The heatmap and radar make that near-tie explicit. No model dominates, and which one looks best depends entirely on the metric you weight: XGBoost wins recall and ROC-AUC, Random Forest wins accuracy and precision, CatBoost wins average precision.

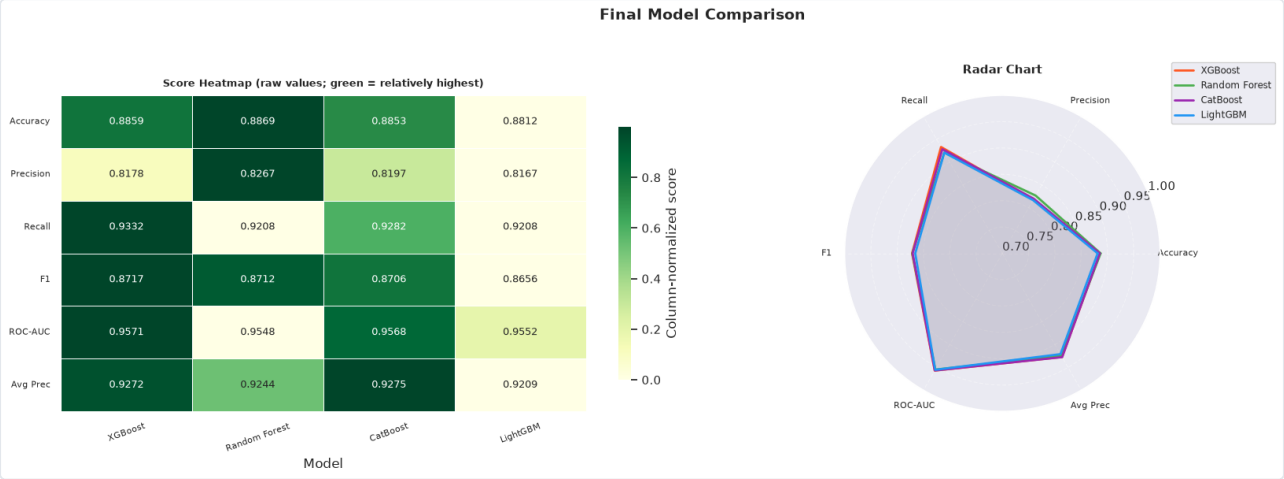

Figure 9. Final comparison. The heatmap (raw scores, green = relatively highest per metric) and the radar chart both show four heavily overlapping profiles. The practical tiebreaker is training cost, not accuracy.

Strengths, limitations, and next steps

Strengths. A pipeline with no leakage, two physically grounded engineered features that the models actually lean on, and a benchmark wide enough to show that the choice of model barely matters on this data.

Limitations. The hyperparameters are fixed defaults, not tuned. The TESS portion is small and harder than Kepler, and the Kepler-heavy mix tilts the model toward Kepler. And mean-decrease-in-impurity importance is known to flatter continuous, high-cardinality features, so I report it as a ranking only, not a weight.

Next steps. Tune each model with nested cross-validation; cross-check the importances with permutation importance; rebalance or reweight the mission mix (or train mission-aware models) to close the TESS gap; and point the trained classifier at the unvetted `CANDIDATE` and `PC` rows I set aside, which is where a triage tool actually earns its keep.

5 Conclusion

A single classifier trained on the merged Kepler and TESS catalogs can tell you real planets from false positives at about 0.955 ROC-AUC, and it does it with features you can actually interpret physically. The four ensembles I tested are interchangeable on accuracy. What carries the discriminating power is the engineered radius ratio and transit duty cycle, not the algorithm, and both rank in the top three predictors for every model. The clearest practical takeaway is the speed gap: LightGBM matches the best accuracy while training roughly ninety times faster than XGBoost.

The honest weakness is the per-mission gap. The model reads Kepler much better than TESS, which comes straight from the Kepler-heavy training mix and the noisier stars TESS looks at. Closing that gap, whether through mission-aware reweighting or just a larger TESS sample, is the most useful thing to try next, along with finally turning the classifier loose on the thousands of still-unvetted candidates that motivated the project in the first place.

6 References

1. J. Winn, "Transits and Occultations," in *Exoplanets*, S. Seager, Ed. Tucson, AZ: Univ. of Arizona Press, 2010, pp. 55-77.
2. W. J. Borucki et al., "Kepler Planet-Detection Mission: Introduction and First Results," *Science*, vol. 327, no. 5968, pp. 977-980, 2010.
3. G. R. Ricker et al., "Transiting Exoplanet Survey Satellite (TESS)," *Journal of Astronomical Telescopes, Instruments, and Systems*, vol. 1, no. 1, 014003, 2015.

4. A. Santerne et al., "SOPHIE velocimetry of Kepler transit candidates," *Astronomy & Astrophysics*, vol. 587, A64, 2016.
5. C. J. Shallue and A. Vanderburg, "Identifying Exoplanets with Deep Learning," *The Astronomical Journal*, vol. 155, no. 2, 94, 2018.
6. S. E. Thompson et al., "Planetary Candidates Observed by Kepler. VIII. A Fully Automated Catalog," *The Astrophysical Journal Supplement Series*, vol. 235, no. 2, 38, 2018.
7. R. L. Akeson et al., "The NASA Exoplanet Archive: Data and Tools for Exoplanet Research," *Publications of the Astronomical Society of the Pacific*, vol. 125, no. 930, pp. 989-999, 2013. Data: *exoplanetarchive.ipac.caltech.edu*.
8. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
9. L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001.
10. T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785-794.
11. G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 3146-3154.
12. L. Prokhorenkova et al., "CatBoost: Unbiased Boosting with Categorical Features," in *Advances in Neural Information Processing Systems 31*, 2018, pp. 6638-6648.